DAY 02 | POST-READ

Functions, Data and Files: Recap and Beyond

What we covered today, plus ideas to stretch your understanding.

Prepared by **Ahmed** | linkedin.com/in/ahmed-al-imaad

Part A: Summary of today

Topic	What to remember
def and return	def name(params): defines a function. return sends a value back. No return means None.
Defaults and keywords	def f(x, rate=18). Call with f(10, rate=5) for clarity.
*args / **kwargs	Collect any number of positional or named arguments.
list	Ordered, changeable. append, remove, slicing, comprehensions.
tuple	Ordered, unchangeable. Great for fixed records and unpacking.
set	Unique items. Fast membership checks. Union and intersection.
dict	Key value pairs. Use get() for safe lookups and items() to loop.
Files	with open(path, mode). r read, w write, a append.
CSV and JSON	csv.DictReader / DictWriter for tables. json.load / json.dump for nested data.

Part B: Going beyond today

1. Mutable default arguments: a classic trap

Never use a list or dict as a default value. It is created once and shared between calls. Use None instead.

```
def add_item(item, basket=None):
    if basket is None:
        basket = []
    basket.append(item)
    return basket
```

2. Dictionary and set comprehensions

```
names = ["ana", "ken", "ana"]
lengths = {n: len(n) for n in names}   # {'ana': 3, 'ken': 3}
unique = {n.upper() for n in names}      # {'ANA', 'KEN'}
```

3. Sorting records with key=

```
students = [{"n": "A", "m": 70}, {"n": "B", "m": 90}]
top = sorted(students, key=lambda s: s["m"], reverse=True)
```

4. Copying is not the same as assigning

b = a makes two names for the same list. Change b and a changes too. Use a.copy() or list(a) for a shallow copy, and copy.deepcopy for nested data.

5. pathlib: modern file paths

```
from pathlib import Path
p = Path("data") / "marks.csv"
p.parent.mkdir(exist_ok=True)
print(p.exists(), p.suffix)
```

6. Type hints make functions self-documenting

```
def average(marks: list[float]) -> float:
    return sum(marks) / len(marks)
```

Type hints do not change how the code runs, but editors and tools like FastAPI (Day 5) use them heavily.

Part C: Check yourself

- What is the difference between a parameter and an argument?
- When would you choose a tuple over a list?
- How do you safely read a key that may not exist in a dict?
- Why does json.dump keep numbers as numbers but CSV does not?
- What does the with statement do for you?

Part D: Practice ideas for tonight

- Write a function that returns the word frequency of a sentence as a dict.
- Read a CSV of expenses and print the total per category.
- Save your favourite movies as JSON, then load and print them sorted by year.

Coming up on Day 3: classes and objects, inheritance, exception handling, SOLID principles, NumPy, modules, decorators and generators.

Otsukaresama deshita (お疲れ様でした): thank you for your hard work today